

University of Chicago
Department of Computer Science

MASTER'S THESIS



Mirza Mehmedagic

The Zig-Zag Pseudorandom Generator

Advisor: William Hoza, PhD

Contents

1	Introduction	1
1.1	Main Result	3
2	Preliminaries	3
3	Expander Graphs	4
3.1	Labelings	5
3.2	Explicitness	6
3.3	Basic Graph Products	6
4	The Zig-Zag PRG	7
5	The Zig-Zag Product	9
6	Singular-Value Approximation	14
7	Fooling Permutation Branching Programs	17
8	Comparison to the INW Generator	20

Acknowledgments

I would firstly like to thank my advisor for his valuable comments and suggestions. His extensive experience with pseudorandomness and derandomization was a great aid to this project.

I would also like to thank the other members of my defense committee and all the professors that have ever taught or mentored me at the University of Chicago.

Finally, I would like to thank my family. I would not be here today without their support and love.

Abstract

Two seminal works proved that the problem of undirected s - t connectivity is decidable deterministically in logarithmic space. Reingold originally proved this fact [Rei08] while Rozenman and Vadhan gave an alternative proof [RV05]. These two proofs remain the only ones demonstrating this fact.

Rozenman and Vadhan use the iterated application of the derandomized square in their proof. They also show that this product could be used to analyze the Impagliazzo-Nisan-Wigderson (INW) pseudorandom generator [INW94]. Even before Rozenman and Vadhan’s work, the INW generator had been used to fool various models of computation, but many newer proofs that the INW generator fools other models of computation often hinge on analysis through the derandomized square product. For instance, it fools unbounded-width permutation branching programs with a single accept state, with nearly optimal seed length [HPV21].

Reingold’s algorithm relies on the iterated application of a different graph product called the zig-zag product, previously introduced in a work [RVW00] by Reingold, Vadhan, and Wigderson. The zig-zag product was later shown to similarly have pseudorandom walk properties on regular graphs [RTV06].

In this thesis, we describe the construction of a “zig-zag pseudorandom generator.” Prior work had already described a pseudorandom walk generator based on the zig-zag product, which was used to visit all vertices in a connected undirected graph [RTV06]. Our contribution is that this “zig-zag pseudorandom generator” can also (like the INW generator) be used to generate pseudorandom bits that fool permutation branching programs with a single accept state, with a seed length of $O(\log n \cdot (\log \log n + \log(1/\epsilon)))$.

The generator is constructed without reference to the zig-zag product though it mimics the iterative construction in Reingold’s algorithm under the hood. Our analysis will make use of the zig-zag product’s spectral expansion properties and recently developed singular-value approximation techniques of random walk matrices [APPSV23].

1 Introduction

Intuitively, pseudorandom generators are meant to take a short string of truly random $\{0, 1\}$ -valued bits (called the seed) as input and produce a longer string of bits that are “pseudorandom” or “appear” random to some function or class of functions.

Definition 1.1. Let $\mathcal{F}$ be a class of functions mapping $\{0, 1\}^n \rightarrow \mathbb{R}$. We say that a distribution X over $\{0, 1\}^n$ ε -fools $\mathcal{F}$ if for all $f \in \mathcal{F}$, we have

$$\left| \mathbb{E}_{x \sim X} [f(x)] - \mathbb{E}_{x \sim U_n} [f(x)] \right| = |\mathbb{E}[f(X)] - \mathbb{E}[f(U_n)]| \leq \varepsilon.$$

where U_n is the uniform distribution on the set $\{0, 1\}^n$

Definition 1.2. Let $\mathcal{F}$ be a class of functions mapping $\{0, 1\}^n \rightarrow \mathbb{R}$. An ε -pseudorandom generator (or ε -PRG) for $\mathcal{F}$ is a function $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ for which the distribution $G(U_s)$ ε -fools $\mathcal{F}$. We call s the seed length of the pseudorandom generator.

Oftentimes, we may denote the domain or codomain of a PRG to be arbitrary sets that we instead encode as a set of appropriately-sized binary strings.

For many classes $\mathcal{F}$ of functions, it is often the case that there is standard argument involving the probabilistic method to show that there exist non-explicit ε -PRGs for $\mathcal{F}$ with optimal seed length $O(\log(n/\varepsilon))$. Ideally, however, we would like a PRG to both be explicitly and efficiently computable while still having as small a seed length as possible.

In this thesis, we will be interested in the class of functions that can be computed by a model known as branching programs. Branching programs can be thought of as a directed acyclic graph, where at each vertex the program queries one of the input bits and goes to either its successor via an edge labeled 0 or the successor via an edge labeled 1 depending on the result. The fan-out of each vertex must be 2 (and a decision tree is, in fact, the special case where the fan-in is 1). We are particularly interested in read-once branching programs.

Definition 1.3. A width- w length- n (standard-order) read-once branching program B is defined by a start state $v_0 \in [w]$, a sequence of n transition functions $B_1, \dots, B_n : [w] \times \{0, 1\} \rightarrow [w]$, and a set of accept states $V \subseteq [w]$. An input $x \in \{0, 1\}^n$ determines a sequence of states $v_0, v_1, \dots, v_n \in [w]$ by the rule $v_i = B_i(v_{i-1}, x_i)$ for $i > 0$. The output of the program is given by

$$B(x) = \begin{cases} 1 & v_n \in V \\ 0 & v_n \notin V \end{cases}.$$

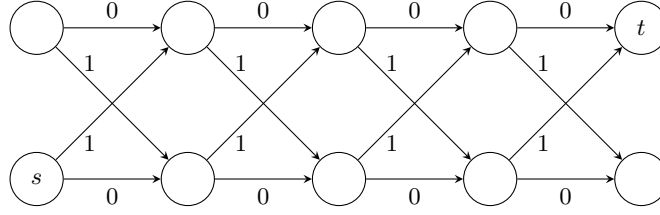
Equivalently, we can think of B as a directed graph with vertices arranged in $n + 1$ layers, $V_0, \dots, V_n$, where $|V_i| = w$. For $i < n$, each vertex $u \in V_i$ has two outgoing edges leading to V_{i+1} , one labeled 0 and the other labeled 1. There is a designated start vertex $v_0 \in V_0$, and there is a set of designated accept vertices $V \subseteq V_n$. An input $x \in \{0, 1\}^n$ is interpreted as a sequence of edge labels, identifying a path $(v_0, v_1, \dots, v_n) \in V_0 \times V_1 \times \dots \times V_n$.

Definition 1.4. A read-once branching program B with layers $V_0, V_1, \dots, V_n$ is said to be

- a regular branching program if each vertex in $V_1, V_2, \dots, V_n$ has two incoming edges.
- a permutation branching program if each vertex in $V_1, V_2, \dots, V_n$ has two incoming edges with distinct labels.

Even with constant width, branching programs are a relatively powerful model of computation. Many functions that are hard for other models of computation (e.g., constant-depth AC^0 circuits) can be computed by constant-width branching programs. The parity function is a classic example of this phenomenon.

Example 1.5. A width-2 length-4 permutation ROBP that computes $x_1 \oplus x_2 \oplus x_3 \oplus x_4$, the parity function on 4 bits, is given by the graph



where s is the start vertex and t is the only accept vertex.

While designing explicit algorithms that take some number of truly (uniformly) random bits to generate ideally exponentially more bits that “appear” random (e.g., to various classes of ROBPs) is a worthy task in and of itself, PRGs have numerous applications in derandomization and cryptography. For instance, if we would like to simulate randomness in computation efficiently, we can deterministically generate pseudorandom strings from all possible seeds of such a suitable PRG and use these strings instead of random ones.

As a more concrete example, let us say that A is a randomized log-space algorithm that decides some language of binary strings. This means that the language is in the class BPL, the class of languages that can be decided by a Turing machine that (on inputs of length n) halts, uses $O(\log n)$ space, and $\text{poly}(n)$ read-once random bits.

If σ is an input of length n that we feed into A , then the function $f(x) := A(\sigma, x)$ on the random bits used by A can be computed by a width- N length- N standard-order ROBP B for a suitable value $N = \text{poly}(n)$. Each state of the branching program encodes one of the at most $\text{poly}(n)$ configurations of the machine A .

So, if we wish to derandomize A , one method is to design an explicitly computable ε -PRG that fools standard-order ROBPs. This is sufficient because the function $f(x)$ of the random bits x of A can be computed by the standard-order ROBP B . Simulating B on all pseudorandom strings from all possible seeds (and taking a deterministic majority vote) can be done with $O(\log n)$ space and, with small ε , is nearly as good as supplying B with truly random bits (and leaving the outcome to chance).

For these reasons, the construction of pseudorandom generators for various classes of ROBPs is a well-studied field. For instance, the Nisan-Zuckerman generator is best employed when $w \gg n$ (i.e., for either wide or short branching programs).

Theorem 1.6 ([NZ96]). *Let $b > 0$ and $c \geq 1$ be constants. Then, for all $w \in \mathbb{N}$, there is an explicit $2^{-\log^{1-b} w}$ -PRG for width- w length- $(\log^c w)$ standard-order ROBPs with seed length $O(\log w)$.*

Nisan’s generator [Nis92] and later the Impagliazzo-Nisan-Wigderson (INW) generator [INW94] are other classics example, best employed when $w \ll n$ (i.e., for narrow branching programs) or $w = n$. Both generators work to prove the following theorem.

Theorem 1.7. *For every $n, w \in \mathbb{N}$ and every $\varepsilon > 0$, there is an explicit ε -PRG for width- w length- n standard-order with seed length $O(\log n \cdot \log(nw/\varepsilon))$.*

Nisan’s PRG and the INW generator follows similar intuition, but the details in their exact constructions are different. The INW generator has some advantages over Nisan’s PRG, not the least of which is that the INW generator has turned out to be more flexible and amenable to analysis in other models.

Largely due to the lack of progress on designing PRGs for general length- n branching programs since the work of Nisan [Nis92], restricted classes of branching programs have received attention.

For example, with regards to regular branching programs, the following was proven by Braverman, Rao, Raz, and Yehudayoff.

Theorem 1.8 ([BRRY14]). *For every $n, w \in \mathbb{N}$ and $\varepsilon > 0$, there is an explicit ε -PRG for width- w length- n standard-order regular ROBPs with seed length $O(\log n \cdot (\log w + \log \log n + \log(1/\varepsilon)))$.*

Regular branching programs are of particular interest because general branching programs can be reduced to regular branching programs with some blowup in the width.

Theorem 1.9 ([LPV23]). *Let B be a width- w length- n standard-order ROBP computing some function $f : \{0, 1\}^n \rightarrow \{0, 1\}$. Then, there exists a width- $O(wn)$ length- n standard-order regular ROBP B' computing f .*

This further motivates the study of wide branching programs, ones where $w \gg n$, since sacrificing width for “nicer” properties like regularity is often more amenable to analysis.

1.1 Main Result

In particular, for the main theorem, we further restrict from regular branching programs to permutation branching programs but allow them to be unbounded in width. What is interesting about this model of computation is that the usual probabilistic method fails [HPV21]. For instance, a counting argument fails since unbounded-width permutation branching programs can compute doubly-exponentially many distinct functions (even with a single accept state).

So, finding an explicit PRG or explicit PRGs for this model is an intriguing task. As it turns out, the $\log w$ term in the seed length in theorem 1.8 is not necessary if we restrict the to permutation branching programs, which means that the width can be unbounded.

Theorem 1.10. *For every $n \in \mathbb{N}$ and $\varepsilon > 0$, there is an explicit ε -PRG for arbitrary-width, length- n permutation branching programs with a single accept state. This PRG has seed length $O(\log n \cdot (\log \log n + \log(1/\varepsilon)))$.*

Theorem 1.10 was originally proven by Hoza, Pyne, and Vadhan [HPV21] using the INW generator. They also demonstrated that the seed length was nearly as any ε -PRG for unbounded-width permutation branching programs must have seed length at least $\Omega(\log n \cdot \log(1/\varepsilon))$.

Analysis of the INW generator was reframed by Rozenman and Vadhan [RV05] in terms of a pseudorandom walk on an iterated graph product relying on the derandomized square product. In that work, Rozenman and Vadhan introduced the derandomized square product and used it to give another proof of Reingold’s seminal theorem that there exists a deterministic log-space algorithm for deciding the undirected s - t connectivity problem [Rei08].

Hoza, Pyne, and Vadhan’s analysis also involved unit-circle approximations of random walk matrices. The theory of unit-circle approximation [AKMPSV20] has more generally been used to understand fixed-length walks on graph products like the derandomized square product, whereas Rozenman and Vadhan focused on how the derandomized square product preserved the connected components of the original graph.

We will present an alternative proof of Theorem 1.10 based on a “zig-zag pseudorandom generator.” A pseudorandom walk generator based on the zig-zag product had already been considered prior to this [RTV06]. This PRG, as it turns out, is related to Reingold’s original log-space algorithm for deciding the undirected s - t connectivity problem. In that algorithm, Reingold employs an iterated graph product relying on the zig-zag product instead.

The theory of singular-value approximation [APPSV23] will be employed in a similar manner here. Reingold focused on how the zig-zag product preserved the connected components of some original graph (i.e., how quickly a random walk on the graph product can mix to a uniform distribution compared to the original graph), but we are interested in comparing fixed-length random walks on a zig-zag product to those on the original graph.

2 Preliminaries

The set of positive integers is denoted by $\mathbb{N}$, and the set of non-negative integers is denoted by $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$.

Unless otherwise noted, $\|\cdot\|$ is always taken to be the standard Euclidean norm. Take J_N to be the N -by- N matrix in which all entries are $1/N$, and take I_N to be the N -by- N identity matrix. If the subscripts are omitted from either matrix, we take them to be a appropriate size given the situation.

For a directed multigraph G , we define its adjacency matrix A_G by

$$A_G(u, v) = \begin{cases} 1 & \text{there is an edge to } v \text{ from } u \\ 0 & \text{otherwise} \end{cases}$$

If G is regular with degree d , we let $M_G = d^{-1}A_G$ denote the random walk matrix (normalized adjacency matrix) of the graph. We will often abuse notation and write G for M_G , where it is applicable.

The tensor product of two vector $u \in \mathbb{R}^n$ and $v \in \mathbb{R}^m$ is the vector $u \otimes v \in \mathbb{R}^{nm}$ defined by $(u \otimes v)_{(a,b)} = u_a \cdot v_b$. The tensor product of two matrices $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{m \times m}$ is the unique matrix $A \otimes B$ such that $(A \otimes B)(u \otimes v) = (Au) \otimes (Bv)$ for all u, v .

The set $\{1, 2, \dots, N\}$ is denoted by $[N]$. Vectors in $\mathbb{R}^N$ will be used to denote probability distributions on $[N]$. We make use of the following relevant probability distributions over $\{0, 1\}^n$:

- u_N denotes the uniform distribution.
- Ax denotes push-forward distribution of a probability distribution x on $[N]$ by the linear transformation A (an N -by- N matrix).

The set of all binary strings of length n will be denoted by $\{0, 1\}^n$, and the set of all binary strings of finite length will be denoted by $\{0, 1\}^*$. We make use of the following relevant probability distributions over $\{0, 1\}^n$:

- U_n denotes the uniform distribution.
- $f(X)$ denotes push-forward distribution of a probability distribution X on a finite set S by the function $f : S \rightarrow \{0, 1\}^n$.

3 Expander Graphs

Before constructing the zig-zag PRG, the basic theory of expander graphs must be developed. The expansion parameter of a graph G essentially capture the notion of how “well-connected” G is.

Definition 3.1. Let G be a regular directed multigraph on N vertices. Then, the spectral expansion parameter is

$$\lambda(G) = \max_{x \perp u_N} \frac{\|Gx\|}{\|x\|} \in [0, 1].$$

Here, the maximum is taken over all non-zero vectors $x \in \mathbb{R}^N$ that are orthogonal to u_N .

This definition essentially encodes how slowly the component of a vector orthogonal to the uniform distribution u_N will disappear under a random walk. $\lambda(G)$ being closer to 0 means that it happens more quickly while $\lambda(G)$ being closer to 1 means that it happens more slowly.

Assuming that G is connected and non-bipartite, u_N is the stationary distribution for a Markov chain modeling a random walk on a regular graph. That is, under these conditions, the uniform distribution is the unique probability vector for which $Gu_N = u_N$ and for which $G^k v \rightarrow u_N$ for all other probability vectors v .

The following is a well-known linear algebraic characterization of the spectral expansion parameter $\lambda(G)$ of any regular directed multigraph G .

Proposition 3.2. For a regular directed multigraph G , $\lambda(G)$ is the square root of second largest eigenvalue of the matrix $G^\top G$.

Proof. Clearly, $G^\top G$ is not only symmetric but also positive semi-definite since

$$v^\top G^\top G v = (Gv)^\top (Gv) = \|Gv\|^2 \geq 0$$

for all $v \in \mathbb{R}^N$.

So, by the spectral theorem of positive semi-definite matrices, all its eigenvalues $\mu_1 \geq \mu_2 \geq \dots \geq \mu_N$ are real numbers in $[0, 1]$, and their corresponding eigenvectors $v_1, \dots, v_N$ form an orthonormal basis in $\mathbb{R}^N$. We can take $v_1 = \sqrt{N}u_N$ because u_N is an eigenvector with eigenvalue 1.

Let $x \in \mathbb{R}^N$ be orthogonal to u_N and thus to v_1 . Then, there exist unique real numbers $\alpha_2, \dots, \alpha_N$ for which $x = \alpha_2 v_2 + \dots + \alpha_N v_N$. As the basis is orthonormal, we have $\|x\|^2 = \alpha_2^2 + \dots + \alpha_N^2$. So,

$$\begin{aligned} \|Gx\|^2 &= x^\top G^\top G x = \left(\sum_{i=2}^N \alpha_i v_i \right)^\top G^\top G \left(\sum_{i=2}^N \alpha_i v_i \right) \\ &= \left(\sum_{i=2}^N \alpha_i v_i^\top \right) \left(\sum_{i=2}^N \alpha_i G^\top G v_i \right) = \left(\sum_{i=2}^N \alpha_i v_i^\top \right) \left(\sum_{i=2}^N \alpha_i \mu_i v_i \right) \\ &= \sum_{i=2}^N \sum_{j=2}^N \alpha_i \alpha_j \mu_j v_i^\top v_j = \sum_{i=2}^N \alpha_i^2 \mu_i \leq \sum_{i=2}^N \alpha_i^2 \mu_2 = \|x\|^2 \mu_2 \end{aligned}$$

yielding $\lambda(G) \leq \sqrt{\mu_2}$. The other direction follows by noticing equality holds above if and only if $x = v_2$. $\square$

In the case that G is undirected, $\lambda(G)$ becomes the second largest eigenvalue of the symmetric matrix G , in absolute value.

Definition 3.3. Let G be a D -regular directed multigraph on N vertices, with $\lambda(G) \leq \lambda < 1$. Then, G is said to be a (N, D) -regular graph and a (N, D, λ) -expander graph.

Due to the following theorem, a large class of graphs are expanders going by Definition 3.1. The gap between $\lambda(G)$ and 1 could be very tiny, but there is always a separation under certain basic conditions.

Theorem 3.4 ([AS00]). *Let G be a connected and non-bipartite (N, D) -regular graph. Then, we have*

$$\lambda(G) \leq 1 - \frac{1}{DN^2}.$$

3.1 Labelings

There are two ways of labeling (expander) graphs that we will make use of throughout this thesis.

Definition 3.5. A *one-way* labeling of a (N, D) -regular graph assigns a label in $[D]$ to each edge (u, v) such that the labels of the outgoing edges of each vertex u are distinct.

If G has a one-way labeling for which the labels of the incoming edges of each vertex u are distinct, this one-way labeling is said to be *consistent*.

If G has a one-way labeling, let $G[u, i]$ denote the vertex v such that (u, v) is labeled i .

Definition 3.6. A *two-way* labeling of a (N, D) -regular graph G assigns two labels in $[D]$ to each edge (u, v) : one as an edge incident to u (the “outgoing label”) and one as an edge incoming to v (the “incoming label”). We require that for every vertex u , the outgoing labels of the outgoing edges of u are distinct and the incoming labels of the incoming edges of u are distinct.

If G has two-way labeling, we define the rotation map $\text{Rot}_G : [N] \times [D] \rightarrow [N] \times [D]$ by letting $\text{Rot}_G(u, i) = (v, j)$ if there is an edge (u, v) with outgoing label i and incoming label j .

It is not hard to see that a graph has a two-way labeling if it has a consistent one-way labeling. The proof of the following lemma more or less follows by definition.

Lemma 3.7. *If G is a (N, D) -regular graph with a consistent one-way labeling, then we can extend G to a graph that has a two-way labeling given by*

$$\text{Rot}_G(v, i) = (G[v, i], i)$$

for all $v \in [N]$ and all $i \in [D]$.

3.2 Explicitness

To be able to use expander graphs for the construction of pseudorandom generators (and for other computational reasons), we would like to have computationally explicit constructions of graphs, given bounds on time and space resources too.

There are many different notions of explicitness depending on what is necessary. We will follow Alon's definitions [Alo21] since they are natural for log-space computation.

Definition 3.8. A family of regular directed multigraphs $\mathcal{G}$ is called explicit if there is a $\text{poly}(ND)$ -time and $O(\log(ND))$ -space deterministic algorithm that outputs the graph $G \in \mathcal{G}$ given the number N of vertices in G and the degree D of G .

Definition 3.9. A family of regular directed multigraphs $\mathcal{G}$ with two-way labelings is called strongly explicit if, for all $G \in \mathcal{G}$, $\text{Rot}_G(v, a)$ can be computed by a $\text{poly}(\log(ND))$ -time and $O(\log(ND))$ -space deterministic algorithm given the number N of vertices in G , the degree D of G , a vertex $v \in [N]$, and an edge label $a \in [D]$.

We will discuss specific strongly explicit expanders that will be used in the construction of the zig-zag PRG later.

3.3 Basic Graph Products

Graph products are ways of obtaining new graphs from one or more other constituent graphs. We define some basic ones in this subsection.

Definition 3.10. Let G be a (N, D) -regular graph with a two-way labeling, and let $t \geq 2$ be an integer. Then, the t -th power G^t of G is the (N, D^t) -regular graph with a two-way labeling whose rotation map Rot_{G^t} is defined by

$$\text{Rot}_{G^t}(v_0, (k_1, k_2, \dots, k_t)) = (v_t, (k'_t, k'_{t-1}, \dots, k'_1))$$

where we define $(v_i, k'_i) = \text{Rot}_G(v_{i-1}, k_i)$ for all $i \in [t]$,

Intuitively, G^t joins vertices u and v for which there exists a walk of length exactly t between u and v in G . Since G is a multigraph, edges in G can have multiplicity strictly greater than one in this walk (i.e., edges can be reused). Also, G^t itself is a multigraph because there can be multiple edges between vertices in G^t (due to multiple distinct walks from one vertex to another).

For the graph products defined in this subsection, we focus on the case where graphs are equipped with two-way labelings. Then, we can get the case where graphs are equipped with consistent one-way labelings by applying Lemma 3.7. For the zig-zag product (defined in a later section), it is non-trivial to show that the definition with a two-way labeling agrees with a definition with a consistent one-way labeling, which we demonstrate by Lemma 5.5.

If we want to notate G^t in the setting of (not necessarily consistent) one-way labelings, this yields a recursive definition given by

$$G^t[v_0, (k_1, k_2, \dots, k_t)] = G[G^{t-1}[v_0, (k_1, k_2, \dots, k_{t-1})], k_t]$$

for all integers $t \geq 2$, where G^1 is the graph G .

The following fact is immediate from the linear algebraic properties of the random walk matrices of graphs.

Fact 3.11. For a (N, D, λ) -expander graph G with a two-way labeling and an integer $t \geq 2$, G^t is a (N, D^t, λ^t) -expander graph with a two-way labeling.

Furthermore, we can also take the power of graphs with consistent one-way labelings to get more graphs with consistent one-way labelings. If each vertex of G has (both its incoming and outgoing) edges labeled with distinct labels in $[D]$, then each vertex of G^t can have its (both its incoming and outgoing) edges labeled with distinct labels in $[D]^t$, a tuple of t labels in $[D]$.

Definition 3.12. Let G be a (N_1, D_1) -regular graph with a two-way labeling, and let H be a (N_2, D_2) -regular graph with a two-way labeling. Then, the tensor product $G \otimes H$ is $(N_1 N_2, D_1 D_2)$ -regular graph with a two-way labeling whose rotation map is defined by

$$\text{Rot}_{G \otimes H}((u, v), (k, \ell)) = ((u', v'), (k', \ell'))$$

where $\text{Rot}_G(u, k) = (u', k')$ and $\text{Rot}_H(v, \ell) = (v', \ell')$.

Intuitively, a single step on the graph $G \otimes H$ involves two independent steps, one on the graph G and one on the graph H , that are reflected coordinate-wise in the vertex set $[N_1] \times [N_2]$.

Again, notating this in the setting of (not necessarily consistent) one-way labelings, this yields a cleaner definition given by

$$(G \otimes H)[(u, v), (k, \ell)] = (G[u, k], H[v, \ell]).$$

The following fact is also mostly immediate from the linear algebraic properties of the random walk matrices of graphs.

Fact 3.13. For a (N_1, D_1, λ_1) -expander graph G with a two-way labeling and (N_2, D_2, λ_2) -expander graph H with a two-way labeling, $G \otimes H$ is a $(N_1 N_2, D_1 D_2, \max\{\lambda_1, \lambda_2\})$ -expander graph with a two-way labeling.

Naturally, consistent one-way labelings are also preserved under tensor products (just as they are under powering).

The bound on the spectral expansion parameter is the only part of this fact that requires some work. To see it, recall that $(G \otimes H)^\top = G^\top \otimes H^\top$ and thus

$$(G \otimes H)^\top (G \otimes H) = (G^\top \otimes H^\top)(G \otimes H) = (G^\top G) \otimes (H^\top H)$$

So, the eigenvectors of $(G \otimes H)^\top (G \otimes H)$ are precisely vectors of the form $u \otimes v$ where u is an eigenvector of $G^\top G$ and v is an eigenvector of $H^\top H$. Letting α and β be the eigenvalues associated to u and v respectively, we get

$$\begin{aligned} (G \otimes H)^\top (G \otimes H)(u \otimes v) &= ((G^\top G) \otimes (H^\top H))(u \otimes v) = (G^\top G u) \otimes (H^\top H v) \\ &= (\alpha u) \otimes (\beta v) = \alpha \beta (u \otimes v) \end{aligned}$$

The square root of the second largest eigenvalue of $G \otimes H$ is thus either $\lambda(G) \cdot 1$ or $1 \cdot \lambda(H)$ since $\lambda(G)^2$ and $\lambda(H)^2$ are the second largest eigenvalues of G and H , respectively.

4 The Zig-Zag PRG

We now describe the “zig-zag pseudorandom generator” in detail. This PRG will actually be a projection (in terms of the output, as a function) of a function that appears in a sequence of “bigger” pseudorandom generators.

First, we will discuss two operations on functions that we can compose in some order to obtain this sequence of PRGs. The first is an operation analogous to powering for functions.

Definition 4.1. Let $F : A \times S \rightarrow A \times T$, and let $k \geq 2$ be an integer. Then, taking $F^1 = F$, define $F^k : A \times S^k \rightarrow A \times T^k$ inductively

$$F^k(a^{(0)}, (s_1, \dots, s_k)) = (a^{(k)}, (t_1, \dots, t_k))$$

for all $a^{(0)} \in A$ and all $s_1, \dots, s_k \in S$. Here, we define $(a^{(i)}, t_i) = F(a^{(i-1)}, s_i)$ for all $i \in [k]$.

Notice that there is some auxiliary state in A that does not get powered when we operate on G to obtain G^k . Instead, a single such that gets transformed k times throughout each

The second is an operation that relies on a regular graph, and we will call it the “zig-zag operation” on functions. It enlarges the auxiliary state set from A to $A \times S$ while shrinking the rest of the domain (assuming the degree D of the graph is less than the size of S).

Definition 4.2. Let $F : A \times S \rightarrow A \times T$, and let H be a $(|S|, D)$ -regular graph. Then, define $(F \sim H) : (A \times S) \times [D]^2 \rightarrow (A \times T) \times [D]$ by

$$(F \sim H)((v, a), ij) = ((\pi_A \circ G(v, H[a, i]), H^2[a, ij]), \pi_T \circ G(v, H[a, i]))$$

for all $v \in A$, all $a \in S$, and all $i, j \in [D]$. Here, π_A and π_T are the projection operators on $A \times T$ onto A and T (respectively).

In the above definition (and from this point forward), we often omit commas in expressing elements of Cartesian products. This is especially where it might be irrelevant to include them or obvious to exclude them.

For each $k \in \mathbb{N}$, our k -th PRG BIGEN_k will take as input a sequence of k labels in $[D^{16}]$ and a sequence of 16 labels in $[D]$, producing as output a sequence of k labels in $[D^{16}]$ and an additional sequence of 8^k labels in $[D^{16}]$. Since we will be able to fix D to be a power of 2 in our choice of the degree of a regular graph H , we can encode all of these sets as sets of binary strings.

Definition 4.3. Let D be a power of 2, and let H be a (D^{16}, D) -regular graph. We define a sequence $\{\text{BIGEN}_k : k \in \mathbb{N}_0\}$, where $\text{BIGEN}_k : [D^{16}]^k \times [D]^{16} \rightarrow [D^{16}]^k \times [D^{16}]^{8^k}$, inductively by

$$\text{BIGEN}_{k+1} = (\text{BIGEN}_k \sim H)^8$$

where BIGEN_0 is defined to be the identity function on $[1] \times [D^{16}]$.

In the above definition, we omit $[1]$ as the first component to the auxiliary state of BIGEN_k for each $k \in \mathbb{N}$. Since $[D]^{16}$ is equal to $[D^{16}]$, it is possible to apply the zig-zag operation and enlarge the auxiliary state set by a factor from $[D^{16}]^k$ to $[D^{16}]^k \times [D^{16}] = [D^{16}]^{k+1}$ at the start of each iteration.

The component of the input that is in $[D^{16}]^k$ is the auxiliary part of input and the component of that is in $[D]^{16}$ is the main part of the input: these correspond to A and S before the application of the zig-zag operation at the start of each iteration. Similarly, the component of the output that is in $[D^{16}]^k$ is the auxiliary part of the output and the component of the output that is in $[D^{16}]^{8^k}$ is the main part of the output: these correspond to A and T before the application of the zig-zag operation at the start of each iteration.

Definition 4.4. For each $n \in \mathbb{N}$, let $\text{GEN}_n : [D^{16}]^{\log_8 n} \times [D]^{16} \rightarrow \{0, 1\}^n$ be defined by

$$\text{GEN}_n(a_1 \cdots a_{\log_8 n}, i_1 i_2 \cdots i_{16}) = b_1 b_2 \cdots b_n$$

where

$$(a'_1 \cdots a'_{\log_8 n}, s_1 s_2 \cdots s_n) = \text{BIGEN}_n(a_1 \cdots a_{\log_8 n}, i_1 i_2 \cdots i_{16})$$

and, for each $i \in [n]$, b_i is the first bit of s_i .

Essentially, in our restriction above, we ignore the auxiliary part of the output and focus on only the first bit of each item in $[D^{16}]$ in the main part of the output.

Again, since D is a power of 2, this naturally yields a PRG in the style of Definition 1.1, where the domain is a set of binary strings with some fixed seed length.

Fact 4.5. For $D = \text{poly}((\log n)/\varepsilon)$, the seed length of GEN_n is $O(\log n \cdot (\log \log n + \log(1/\varepsilon)))$.

This is exactly what we will need to prove Theorem 1.10.

5 The Zig-Zag Product

To analyze this zig-zag PRG, we rely on iterated graph products with expander graphs. Just as how the derandomized square product can be used to analyze the INW generator, the zig-zag product will be used to analyze the aptly-named zig-zag generator. The zig-zag product of two graphs was originally developed by Reingold, Vadhan, and Wigderson [RVW00].

Definition 5.1. Let G be a (N, K) -regular graph with a two-way labeling, and let H be a (K, D) -regular graph with a two-way labeling. Then, the zig-zag product $G \circledast H$ is defined to be the (NK, D^2) -regular graph whose rotation map is defined by

$$\text{Rot}_{G \circledast H}((v, k), (i, j)) = ((w, \ell), (i', j'))$$

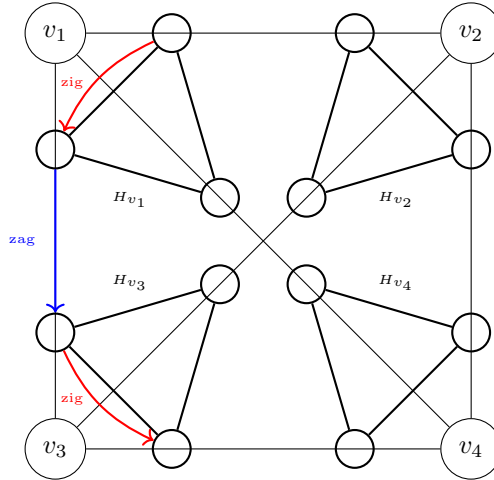
where we compute the following in order:

1. $(k', i') = \text{Rot}_H(k, i)$
2. $(w, \ell') = \text{Rot}_G(v, k')$
3. $(\ell, j') = \text{Rot}_H(\ell', j)$

Intuitively, we can think of one random step in the zig-zag product $G \circledast H$ as involving one random step in a copy of H located at some vertex of G , one deterministic step in G to move between the various copies of H (according to labels), and one random step in another copy of H located at this other vertex of G .

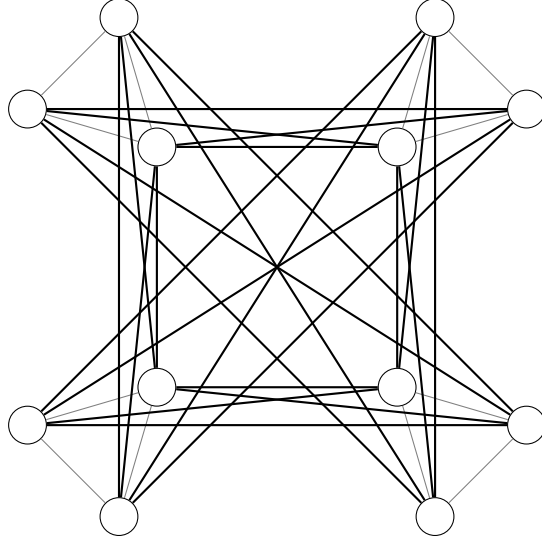
Each copy of H is like a “cloud” that contributes K vertices for each vertex in G (of which there are N). This naturally yields the NK vertices that compose $G \circledast H$.

Example 5.2. If G is the complete graph K_4 on 4 vertices and H is the cycle C_3 on 3 vertices, one step on the graph $G \circledast H$ consists of the three steps outlined below (a zig in copy of H , a zag across copies of H , and a zig in another copy of H).



We omit the labelings of edges here for clarity, but the labels of vertices $\{1, 2, 3\}$ in each copy of H should correspond to (two-way) labels of edges in G for the zag.

As an implicit labeling to employ in the zag, a vertex in a copy of H should correspond to the edge of G it coincides with. The complete zig-zag product $G \circledast H$ with this in mind is drawn below (where the edges in gray are the edges within copies of H , not edges in $G \circledast H$).



This intuitive way of thinking about the zig-zag product can be formalized with the following decomposition of the random walk matrix of $G \mathbin{\text{\textcircled{Z}}} H$ into three matrices that depends on G and H (ultimately a zig, a zag, and another zig).

Lemma 5.3. *Let G be a (N, K) -regular graph, and let H be a (K, D) -regular graph. If we define $P \in \{0, 1\}^{NK \times NK}$ to be the permutation matrix corresponding to the map Rot_G , we have*

$$G \mathbin{\text{\textcircled{Z}}} H = (I_N \otimes H)P(I_N \otimes H) \quad (1)$$

Proof. Let $e_{(v,k)} = e_v \otimes e_k$ be an arbitrary basis vector in $\mathbb{R}^{NK}$. Here, $e_v \in \mathbb{R}^N$ and $e_k \in \mathbb{R}^K$ are the vectors with all zeroes except for a one in the v -th entry and in the k -th entry, respectively.

Define $k'_1, k'_2, \dots, k'_D$ to be the D neighbors of k in H . Define $\text{Rot}_G(v, k'_i) = (w_i, \ell'_i)$ for each $i \in [D]$. Define $\ell_{i1}, \ell_{i2}, \dots, \ell_{iD}$ to be the D neighbors of ℓ'_i in H for each $i \in [D]$.

Note that $\{(w_i, \ell_{ij}) : i, j \in [D]\}$ are precisely the D^2 neighbors of (v, k) in $G \mathbin{\text{\textcircled{Z}}} H$. Thus, by properties of the random walk matrix of graphs, we get

$$(G \mathbin{\text{\textcircled{Z}}} H)e_{(v,k)} = \frac{1}{D^2} \sum_{i=1}^D \sum_{j=1}^D e_{(w_i, \ell_{ij})}$$

On the other hand, we first have

$$\begin{aligned} (I_N \otimes H)e_{(v,k)} &= (I_N \otimes H)(e_v \otimes e_k) = e_v \otimes (He_k) \\ &= e_v \otimes \left(\frac{1}{D} \sum_{i=1}^D e_{k'_i} \right) = \frac{1}{D} \sum_{i=1}^D e_v \otimes e_{k'_i} = \frac{1}{D} \sum_{i=1}^D e_{(v, k'_i)} \end{aligned}$$

By the definition of P , we know $Pe_{(v, k'_i)} = e_{(w_i, \ell'_i)}$ for all $i \in [D]$. So, we then have

$$P(I_N \otimes H)e_{(v,k)} = P \left(\frac{1}{D} \sum_{i=1}^D e_{(v, k'_i)} \right) = \frac{1}{D} \sum_{i=1}^D Pe_{(v, k'_i)} = \frac{1}{D} \sum_{i=1}^D e_{(w_i, \ell'_i)}$$

Finally, we have

$$\begin{aligned} (I_N \otimes H)P(I_N \otimes H)e_{(v,k)} &= (I_N \otimes H) \left(\frac{1}{D} \sum_{i=1}^D e_{(w_i, \ell'_i)} \right) = \frac{1}{D} \sum_{i=1}^D (I_N \otimes H)e_{(w_i, \ell'_i)} \\ &= \frac{1}{D} \sum_{i=1}^D (I_N \otimes H)(e_{w_i} \otimes e_{\ell'_i}) = \frac{1}{D} \sum_{i=1}^D e_{w_i} \otimes (He_{\ell'_i}) \\ &= \frac{1}{D} \sum_{i=1}^D e_{w_i} \otimes \left(\frac{1}{D} \sum_{j=1}^D e_{\ell_{ij}} \right) = \frac{1}{D^2} \sum_{i=1}^D \sum_{j=1}^D e_{w_i} \otimes e_{\ell_{ij}} = \frac{1}{D^2} \sum_{i=1}^D \sum_{j=1}^D e_{(w_i, \ell_{ij})} \end{aligned}$$

Two NK -by- NK matrices are equivalent as operators if and only if they act the same on the complete set of basis vectors $\{e_{(v,k)} : v \in [N], k \in [K]\}$. Thus, $G \circledast H = (I_N \otimes H)P(I_N \otimes H)$. $\square$

The matrix P in this decomposition consists of only zeroes and ones because an input (v, k) to the rotation map of G has precisely one output. This is an indication that the zag substep of a random step on the zig-zag product is indeed deterministic. That is, the label in $[K]$ of the vertex of the current copy of H is the outgoing label in $[K]$ of the edge of G that must be taken in this substep.

The zig-zag product $G \circledast H$ admits nice spectral properties if G and H do as well. The following result is a nice bound that demonstrates this fact though it is not part of the proof of Theorem 1.10.

Lemma 5.4 ([RVW00]). *Let G be a (N, K, λ) -expander graph with a two-way labeling, and let H be a (K, D, μ) -expander graph with a two-way labeling, then $G \circledast H$ is a $(ND, d^2, g(\lambda, \mu))$ -expander graph, where*

$$f(\lambda, \mu) = \frac{1}{2}(1 - \mu^2)\lambda + \frac{1}{2}\sqrt{(1 - \mu^2)^2\lambda^2 + 4\mu^2} \leq \lambda + \mu$$

and the function f is monotonically increasing in λ and μ .

Just as we did for the other graph products, we can write out a one-way labeling of the zig-zag product $G \circledast H$. In fact, we will write out *consistent* one-way labeling of $G \circledast H$ (assuming a consistent one-way labeling of G) and show that it follows from the original definition.

Lemma 5.5. *Let G be a (N, K) -regular graph with a consistent one-way labeling, and let H be a (K, D) -regular graph with a consistent one-way labeling. Then, for any $v \in [N]$, $k \in [K]$, and $i, j \in [D]$, we have*

$$(G \circledast H)[(v, k), (i, j)] = (G[v, H[k, i]], H^2[k, (i, j)]) \quad (2)$$

by the definition of $G \circledast H$, and $G \circledast H$ has a consistent one-way labeling itself.

When we take a step along the first coordinate of the zig-zag product (i.e., the zag substep, which uses the rotation map of G), the second coordinate of the zig-zag product should not change with a consistent one-way labeling. Such a change may happen with a two-way labeling which cannot be made into a consistent one-way labeling, which is why this assumption is necessary.

Proof of Lemma 5.5. Notice by definition that (v, k) is adjacent to (w, ℓ) in $G \circledast H$ if and only if we have all of

1. $\text{Rot}_H(k, i) = (k', i')$
2. $\text{Rot}_G(v, k') = (w, \ell')$
3. $\text{Rot}_H(\ell', j) = (k', j')$

for some $k', \ell' \in [K]$ and $i, j, i', j' \in [D]$. As G and H are equipped with consistent one-way labelings, we deduce that $i = i'$ and $j = j'$ and $k' = \ell'$.

Now, since $k' = H[k, i]$ and $\ell = H[\ell', j]$, we have $\ell = H^2[k, (i, j)]$. So, the second coordinate of (2) is correct. We also have $w = G[v, k'] = G[v, H[k, i]]$. So, the first coordinate of (2) is correct.

Finally, to prove that $G \circledast H$ has a consistent one-way labeling, fix (i, j) and suppose that

$$(G \circledast H)[(v, k), (i, j)] = (G \circledast H)[(v', k'), (i, j)].$$

It suffices to show that $v = v'$ and $k = k'$. Indeed, by decomposing (2), we have

$$G[v, H[k, i]] = G[v', H[k', i]] \quad \text{and} \quad H[H[k, i], j] = H[H[k', i], j]$$

As H has a consistent one-way labeling, we get $H[k, i] = H[k', i]$. Again, using the fact that H has a consistent one-way labeling, this implies that $k = k'$.

Since $k = k'$, $H[k, i] = H[k', i]$. As G has a consistent one-way labeling, we get $v = v'$ from $G[v, H[k, i]] = G[v', H[k', i]]$. $\square$

Now, the sequence $\{\text{BIGEN}_k : k \in \mathbb{N}\}$ of PRGs from Definition 4.3 is closely tied to the iterative construction of a graph product used in Reingold's algorithm given an initial graph G and an auxiliary expander graph H . This will form the basis for our analysis of the zig-zag PRG.

For the purposes of later analysis, the initial graph G should be thought of a permutation branching program, or its corresponding layered acyclic graph outlined in Definition 1.3 more specifically. By definition, this graph should have a consistent one-way labeling already.

To make this graph regular, we can add two edges (labeled 0 to 1) from the i -th vertex in the final layer to the i -th vertex of the first layer. We call this "wraparound." In order for the degree of the graph to be D^{16} instead of 2, we can duplicate each edge $D^{16}/2$ times and have the original label of each edge be the first bit of the new label.

Definition 5.6. Let G be a (N, D^{16}) -regular graph and H be a (D^{16}, D) -regular graph, each with a consistent one-way labeling. Then, define a sequence $\{\mathcal{T}^k(G, H) : k \in \mathbb{N}_0\}$ inductively in the following way:

- $\mathcal{T}^0(G, H) = G$
- $\mathcal{T}^k(G, H) = (\mathcal{T}^{k-1}(G, H) \otimes H)^8$ for each $k \in \mathbb{N}$

Let $\mathcal{T}_n(G, H)$ denote the graph $\mathcal{T}^{\log_8 n}(G, H)$ for a chosen number n .

For each $k \in \mathbb{N}$, $\mathcal{T}^k(G, H)$ is a D^{16} -regular graph on the vertex set $[N] \times [D^{16}]^k$. The component of the vertex set in $[D^{16}]^k$ will be called the auxiliary component of some vertex $v \in [N] \times [D^{16}]^k$, matching the language of the first components of the input and the output to BIGEN_k .

We can extend the consistent one-way labeling of the zig-zag product by directly writing out the 8th power to understand the iterative step in Definition 5.6. That is, we have

$$\mathcal{T}^k(G, H)[(v, a), i_1 i_2 \cdots i_{16}] = (\mathcal{T}^{k-1}(G, H)^8[v, H[a, i_1] H^3[a, i_1 i_2 i_3] H^{15}[a, i_1 i_2 \cdots i_{15}], H^{16}[a, i_1 i_2 \cdots i_{16}]])$$

for each $k \in \mathbb{N}$.

There is an intimate connection between the sequence $\{\text{BIGEN}_k : k \in \mathbb{N}\}$ of PRGs defined earlier and the sequence $\{\mathcal{T}^k(G, H) : k \in \mathbb{N}\}$ of these graphs, if we construct them with the same auxiliary graph H .

Theorem 5.7. Let G be a (N, D^{16}) -regular graph and H be a (D^{16}, D) -regular graph, each with a consistent one-way labeling. Then, for all $k \in \mathbb{N}$, we have

$$\mathcal{T}^k(G, H)[(v, a_1, \dots, a_k), i_1 i_2 \cdots i_{16}] = (G^{8^k}[v, \pi_2 \circ \text{BIGEN}_k(a_1 \cdots a_k, i_1 i_2 \cdots i_{16})], \pi_1 \circ \text{BIGEN}_k(a_1 \cdots a_k, i_1 i_2 \cdots i_{16}))$$

where π_1 and π_2 are the projection operators on $[D^{16}]^k \times [D^{16}]^{8^k}$ onto $[D^{16}]^k$ and $[D^{16}]^{8^k}$ (respectively), the auxiliary and main parts of the output.

Proof. Throughout this proof, let $G_k = \mathcal{T}^k(G, H)$, for simplicity. We proceed by induction on k .

The base case follows by first writing out a formula for BIGEN_1 . Since BIGEN_0 is the identity function on $[1] \times [D^{16}]$, we have

$$\begin{aligned} (\text{BIGEN}_0 \sim H)((1, a), i_1 i_2) &= ((\pi_{[1]} \circ \text{BIGEN}_0(1, H[a, i_1]), H^2[a, i_1 i_2]), \pi_{[D^{16}]} \circ \text{BIGEN}_0(1, a)) \\ &= ((1, H^2[a, i_1 i_2]), H[a, i_1]). \end{aligned}$$

where $a \in [D^{16}]$ and $i_1, i_2 \in [D]$.

From now on, we can omit the first component of the auxiliary state (the element of $[1]$). So, we write $(\text{BIGEN}_0 \sim H)(a, i_1 i_2) = (H^2[a, i_1 i_2], H[a, i_1])$ instead.

Powering this function 7 more times yields various points along a walk on H according to the labels $i_1, i_2, \dots, i_{16} \in [D]$. That is, $(\text{BIGEN}_0 \sim H)(a, i_1 i_2) = (H^2[a, i_1 i_2], H[a, i_1])$ and

$$\begin{aligned} &(\text{BIGEN}_0 \sim H)(H^{2j}[a, i_1 i_2 \cdots i_{2j-1} i_{2j}], i_{2j+1} i_{2j+2}) \\ &= (H^2[H^{2j}[a, i_1 i_2 \cdots i_{2j-1} i_{2j}], i_{2j+1} i_{2j+2}], H[H^{2j}[a, i_1 i_2 \cdots i_{2j-1} i_{2j}], i_{2j+1}]) \\ &= (H^{2j+2}[a, i_1 i_2 \cdots i_{2j+1} i_{2j+2}], H^{2j+1}[a, i_1 i_2 \cdots i_{2j}, i_{2j+1}]) \end{aligned}$$

for each $j \in [7]$. So, we get

$$\text{BIGEN}_1(a, i_1 i_2 \cdots i_{16}) = (H^{16}[a, i_1 i_2 \cdots i_{16}], H[a, i_1] H^3[a, i_1 i_2 i_3] H^{15}[a, i_1 i_2 \cdots i_{15}])$$

Finally, by the definition of the (8th power of) zig-zag product, we get

$$\begin{aligned} G_1[(v, a), i_1 i_2 \cdots i_{16}] &= (G_0(\mathbb{Z}H)^8[(v, a), i_1 i_2 \cdots i_{16}] \\ &= (G(\mathbb{Z}H)^8[(v, a), i_1 i_2 \cdots i_{16}] \\ &= (G^8[v, H[a, i_1] H^3[a, i_1 i_2 i_3] H^{15}[a, i_1 i_2 \cdots i_{15}]], H^{16}[a, i_1 i_2 \cdots i_{16}]) \\ &= G^8[v, \pi_2 \circ \text{BIGEN}_1(a, i_1 i_2 \cdots i_{16}), \pi_1 \circ \text{BIGEN}_1(a, i_1 i_2 \cdots i_{16})]). \end{aligned}$$

For the inductive case, assume that our claim holds true for G_k as our inductive hypothesis. That is, for some $k \in \mathbb{N}$, we have

$$G_k[(v, a_1, \dots, a_k), i_1 i_2 \cdots i_{16}] = (G^{8^k}[v, \pi_2 \circ \text{BIGEN}_k(a_1 \cdots a_k, i_1 i_2 \cdots i_{16})], \pi_1 \circ \text{BIGEN}_k(a_1 \cdots a_k, i_1 i_2 \cdots i_{16}))$$

where $v \in [N]$, each $a_j \in [D^{16}]$, and each $i_j \in [D]$.

In trying to determine what BIGEN_{k+1} does to an input $(a_1 a_2 \cdots a_k a_{k+1}, i_1 i_2 \cdots i_{16})$, we first define $a'_{k+1} = H^{16}[a_{k+1}, i_1 i_2 \cdots i_{16}]$ and

$$e_j = H^{2j-1}[a_{k+1}, i_1 i_2 \cdots i_{2j-1}]$$

for each $j \in [8]$.

Write $a^{(0)} = a_1 \cdots a_k$, and denote a_{k+1} as a “walk” on H starting from a_{k+1} by $H^0[a_{k+1}, i_1 i_2 \cdots i_0]$. Then, letting $(a^{(j)}, e^{(j)}) = \text{BIGEN}_k(a^{(j-1)}, e_j)$ for each $j \in [8]$, we have

$$\begin{aligned} (\text{BIGEN}_k \sim H)((a^{(j-1)}, H^{2j-2}[a_{k+1}, i_1 i_2 \cdots i_{2j-2}]), i_{2j-1} i_{2j}) \\ = ((a^{(j)}, H^{2j}[a_{k+1}, i_1 i_2 \cdots i_{2j}]), e^{(j)}) \end{aligned}$$

and

$$G_k[(v, a^{(j-1)}), e_j] = (G^{8^k}[v, e^{(j)}], a^{(j)})$$

for each $j \in [8]$. The latter follows by the inductive hypothesis.

With all these labels defined, we now can write

$$\text{BIGEN}_{k+1}(a^{(0)} a_{k+1}, i_1 i_2 \cdots i_{16}) = (a^{(8)} a'_{k+1}, e^{(1)} e^{(2)} \cdots e^{(8)}).$$

By multiple applications of the inductive hypothesis, we can also write

$$\begin{aligned} G_k^8[(v, a^{(0)}), e_1 \cdots e_8] &= G_k^7[(G^{8^k}[v, e^{(1)}], a^{(1)}), e_2 \cdots e_8] \\ &= G_k^6[(G^{2 \cdot 8^k}[v, e^{(1)} e^{(2)}], a^{(2)}), e_3 \cdots e_8] \\ &= \cdots = G_k[(G^{7 \cdot 8^k}[v, e^{(1)} e^{(2)} \cdots e^{(7)}], a^{(7)}), e_8] \\ &= (G^{8^{k+1}}[v, e^{(1)} \cdots e^{(8)}], a^{(8)}) \end{aligned}$$

Finally, by the definition of the (8th power of) zig-zag product, we get

$$\begin{aligned} G_{k+1}[(v, a_1 \cdots a_k a_{k+1}), i_1 i_2 \cdots i_{16}] \\ &= (G_k(\mathbb{Z}H)^8[(v, a_1 \cdots a_k a_{k+1}), i_1 i_2 \cdots i_{16}] \\ &= (G_k^8[(v, a_1 \cdots a_k), H[a_{k+1}, i_1] H^3[a_{k+1}, i_1 i_2 i_3] \cdots H^{15}[a_{k+1}, i_1 i_2 \cdots i_{15}]], \\ &\quad H^{16}[a_{k+1}, i_1 i_2 \cdots i_{16}]) \\ &= (G_k^8[(v, a_1 \cdots a_k), e_1 \cdots e_8], a_{k+1}) = (G_k^8[(v, a^{(0)}), e_1 \cdots e_8], a'_{k+1}) \\ &= ((G^{8^{k+1}}[v, e^{(1)} \cdots e^{(8)}], a^{(8)}), a'_{k+1}) = (G^{8^{k+1}}[v, e^{(1)} \cdots e^{(8)}], a^{(8)} a'_{k+1}) \\ &= (G^{8^{k+1}}[v, \pi_2 \circ \text{BIGEN}_{k+1}(a_1 \cdots a_k a_{k+1}, i_1 i_2 \cdots i_{16})], \\ &\quad \pi_1 \circ \text{BIGEN}_{k+1}(a_1 \cdots a_k a_{k+1}, i_1 i_2 \cdots i_{16})). \end{aligned}$$

This completes the induction and thus the proof of the claim. $\square$

So, a random starting auxiliary vertex and a random step on $\mathcal{T}^k(G, H)$ is exactly a uniform random input to the generator BIGEN_k . When passing such an input into BIGEN_k , it yields a pseudorandom walk of length 8^k on G in the main part of the output.

The two projections outlined in Definition 4.4 correspond to projecting the pseudorandom step from taking place on $G^n \otimes J$ to taking place on G^n (i.e., ignoring the auxiliary components of the vertices) and to ignoring all but the first bit of the edge label. Thus, a random starting auxiliary vertex and a random step on $\mathcal{T}_n(G, H)$ is exactly a uniform random input to the zig-zag PRG GEN_n .

6 Singular-Value Approximation

In [Rei08], $\mathcal{T}_n(G, H)$ is shown to have preserved the connectivity properties of a graph G , which allowed Reingold to develop his deterministic log-space algorithm for testing the connectivity of a graph G .

$\mathcal{T}_n(G, H)$ and the other graphs in the sequence $\{\mathcal{T}^k(G, H) : k \in \mathbb{N}\}$ also have nice expander properties, assuming a good choice of an auxiliary expander H . In showing that GEN_n fools permutation branching programs, we will need stronger properties than this, but this is still a historic lemma worth discussing.

Lemma 6.1 ([Rei08]). *Let G and H be graphs as in Definition 5.6. If we have $\lambda(H) \leq 0.5$, then we have $\lambda(\mathcal{T}^k(G, H)) \leq \max\{\lambda(\mathcal{T}^{k-1}(G, H))^2, 0.5\}$ for all $k \in \mathbb{N}$.*

Proof. For any $\lambda \in [0, 1]$ and $\mu \in [0, 1]$,

$$\begin{aligned} f(\lambda, \mu) &= \frac{1}{2}(1 - \mu^2)\lambda + \frac{1}{2}\sqrt{(1 - \mu^2)^2\lambda^2 + 4\mu^2} \leq \frac{1}{2}(1 - \mu^2)\lambda + \frac{1}{2}\sqrt{(1 - \mu^2)^2 + 4\mu^2} \\ &= \frac{1}{2}(1 - \mu^2)\lambda + \frac{1}{2}\sqrt{(1 + \mu^2)^2} = \frac{1}{2}(1 - \mu^2)\lambda + \frac{1}{2}(1 + \mu^2) \\ &= \frac{1}{2}(1 - \mu^2)\lambda + 1 - \frac{1}{2}(1 - \mu^2) = \frac{1}{2}(1 - \mu^2)(\lambda - 1) + 1 = 1 - \frac{1}{2}(1 - \mu^2)(1 - \lambda) \end{aligned}$$

where $f(\lambda, \mu)$ is the bound on spectral expansion in Lemma 5.8.

Denote $\lambda = \lambda(G_{i-1})$. Since $\lambda(H) \leq 0.5$, we have

$$\lambda(G_{i-1} \otimes H) \leq 1 - \frac{1}{2}(1 - 0.5^2)(1 - \lambda) = 1 - \frac{3}{8}(1 - \lambda) < 1 - \frac{1}{3}(1 - \lambda)$$

Thus, we have $\lambda(G_{i-1}) < (1 - \frac{1}{3}(1 - \lambda))^8$ by Fact 5.2. When considering the two cases based on the size of λ , the lemma follows.

- If $\lambda < 0.5$, then $\lambda(G_i) < (\frac{5}{6})^8 < 0.5$.
- If $\lambda \geq 0.5$, we that $(1 - \frac{1}{3}(1 - \lambda))^4 \leq \lambda$ and therefore $\lambda(G_i) < \lambda^2$. □

Beyond remaining a good expander if H is one, $\mathcal{T}^k(G, H)$ as a random walk matrix is well-approximated by the tensor of G^{8^k} as a random walk matrix with the random walk matrix of the complete graph on an appropriate number of vertices.

So, the next step of the analysis lies in the recently-developed theory of singular-value (SV) approximation of matrices [APPSV23], which we aim to overview in this section.

Random walk matrices of regular graphs are ultimately doubly-stochastic matrices: that is, each entry is non-negative, and each row and column sum to 1. The most natural notion of SV approximation exists for this specific class of matrices.

Definition 6.2. Let W and $\widetilde{W}$ be doubly-stochastic matrices in $\mathbb{R}^{n \times n}$. Then, $\widetilde{W}$ is an ε -SV normalized approximation of W if for all vectors $x, y \in \mathbb{R}^n$, we have:

$$|x^\top (\widetilde{W} - W)y| \leq \frac{\varepsilon}{4} (||x||^2 - ||W^\top x||^2 + ||y||^2 - ||Wy||^2)$$

We denote this by $\widetilde{W} \stackrel{svn}{\approx}_\varepsilon W$.

SV approximation is stronger than previous notions of spectral approximation considered in the literature, including the standard approximation for directed graphs [CKPPRSV17] and unit-circle approximation for directed graphs [AKMPSV20].

Note the asymmetry between $\widetilde{W}$ and W in this definition: the right-hand side makes a reference to W but $\widetilde{W}$. With this in mind, the use of different test vectors x and y on the left-hand side is crucial for capturing the asymmetric information in $\widetilde{W}$ and W .

Notions of SV approximation exist for arbitrary matrices, and it turns out that we cannot prove some key properties of SV normalized approximation without more general SV approximation. We will only discuss the case where we work with specific degree matrices, formed by summing the rows and columns of a matrix, as part of the approximation.

Definition 6.3. Let A and $\widetilde{A}$ be any matrices in $\mathbb{R}^{m \times n}$. If we let $1_k \in \mathbb{R}^k$ denote the vector of all ones, define $D_{\text{out}} = \text{diag}(A1_n)$ and $D_{\text{in}} = \text{diag}(A^\top 1_m)$. Then, $\widetilde{W}$ is an ε -SV approximation of A if for all vectors $x \in \mathbb{R}^m$ and $y \in \mathbb{R}^n$, we have:

$$|x^\top (\widetilde{A} - A)y| \leq \frac{\varepsilon}{4} (x^\top (D_{\text{out}} - AD_{\text{in}}^+ A^\top)x + y^\top (D_{\text{in}} - A^\top D_{\text{out}}^+ A)y)$$

We denote this by $\widetilde{W} \overset{sv}{\approx}_\varepsilon W$.

Remark 6.4. For doubly-stochastic matrices $\widetilde{W}$ and W in $\mathbb{R}^{n \times n}$, $\widetilde{W} \overset{svn}{\approx}_\varepsilon W$ is equivalent to $\widetilde{W} \overset{sv}{\approx}_\varepsilon W$, where $D_{\text{in}} = D_{\text{out}} = I_n$.

Many properties of SV approximation are outlined in [APPSV23]. We list some of the necessary ones here.

Fact 6.5. Suppose that, for $\varepsilon \in (0, 0.5)$ and $W_0, \dots, W_\ell$, we have $W_i \overset{svn}{\approx}_{\varepsilon/2\ell} W_{i-1}$ for each $i \in [\ell]$. Then, $W_\ell \overset{sv}{\approx}_\varepsilon W_0$.

Fact 6.6. Suppose that $\widetilde{W}_1 \overset{svn}{\approx}_\varepsilon W_1$ and $\widetilde{W}_2 \overset{svn}{\approx}_\varepsilon W_2$. Then, $\widetilde{W}_1 \widetilde{W}_2 \overset{svn}{\approx}_{\varepsilon + O(\varepsilon^2)} W_1 W_2$.

Fact 6.7. Suppose that $\widetilde{W} \overset{svn}{\approx}_\varepsilon W$ and that P is an arbitrary permutation matrix in $\mathbb{R}^{n \times n}$. Then, $P\widetilde{W} \overset{svn}{\approx}_\varepsilon PW$ and $\widetilde{W}P \overset{svn}{\approx}_\varepsilon WP$.

However, the following lemma is not given too much attention in the original paper, so we prove it here for completeness. This is the part where we will need the more general notion of SV approximation.

Lemma 6.8. Let A and $\widetilde{A}$ be any matrices in $\mathbb{R}^{n \times n}$ with $\widetilde{A} \overset{sv}{\approx}_\varepsilon A$, and let $M \in \{0, 1\}^{N \times N}$. Then, $M \otimes \widetilde{A} \overset{sv}{\approx}_\varepsilon M \otimes A$.

Proof. First, fix any non-negative integers i, j, k, ℓ . Define

$$\widetilde{A}' = \begin{bmatrix} 0^{i \times j} & 0^{i \times n} & 0^{i \times \ell} \\ 0^{n \times j} & \widetilde{A} & 0^{n \times \ell} \\ 0^{k \times j} & 0^{k \times n} & 0^{k \times \ell} \end{bmatrix} \quad \text{and} \quad A' = \begin{bmatrix} 0^{i \times j} & 0^{i \times n} & 0^{i \times \ell} \\ 0^{n \times j} & A & 0^{n \times \ell} \\ 0^{k \times j} & 0^{k \times n} & 0^{k \times \ell} \end{bmatrix}.$$

Then, a simple computation will reveal that the degree matrices are

$$D'_{\text{out}} = \begin{bmatrix} 0^{i \times i} & 0^{i \times n} & 0^{i \times k} \\ 0^{n \times i} & D_{\text{out}} & 0^{n \times k} \\ 0^{k \times i} & 0^{k \times n} & 0^{k \times k} \end{bmatrix} \quad \text{and} \quad D'_{\text{in}} = \begin{bmatrix} 0^{j \times j} & 0^{j \times n} & 0^{j \times \ell} \\ 0^{n \times j} & D_{\text{in}} & 0^{n \times \ell} \\ 0^{\ell \times j} & 0^{\ell \times n} & 0^{\ell \times \ell} \end{bmatrix}.$$

Letting $x = (x_0, x_1, x_2)^\top$ and $v = (y_0, y_1, y_2)^\top$ be of the proper dimensions, we get

$$\begin{aligned} |x^\top (\widetilde{A}' - A')y| &= |x_1^\top (\widetilde{A} - A)y_1| \\ &\leq \frac{\varepsilon}{4} (x_1^\top (D_{\text{out}} - AD_{\text{in}}^+ A^\top)x_1 + y_1^\top (D_{\text{in}} - A^\top D_{\text{out}}^+ A)y_1) \\ &= \frac{\varepsilon}{4} \left(x^\top \begin{bmatrix} 0^{i \times i} & 0^{i \times n} & 0^{i \times k} \\ 0^{n \times i} & D_{\text{out}} - AD_{\text{in}}^+ A^\top & 0^{n \times k} \\ 0^{k \times i} & 0^{k \times n} & 0^{k \times k} \end{bmatrix} x + y^\top \begin{bmatrix} 0^{j \times j} & 0^{j \times n} & 0^{j \times \ell} \\ 0^{n \times j} & D_{\text{in}} - A^\top D_{\text{out}}^+ A & 0^{n \times \ell} \\ 0^{\ell \times j} & 0^{\ell \times n} & 0^{\ell \times \ell} \end{bmatrix} y \right) \\ &= \frac{\varepsilon}{4} (x^\top (D'_{\text{out}} - A'(D'_{\text{in}})^+(A')^\top)x + y^\top (D'_{\text{in}} - (A')^\top (D'_{\text{out}})^+ A')y). \end{aligned}$$

So, $\tilde{A}' \overset{sv}{\approx}_\varepsilon A'$. These matrices are known as “liftings” of $\tilde{A}, A$, and so we have demonstrated that SV approximation is closed under arbitrary liftings.

Finally, let $\tilde{A}_1, A_1$ and $\tilde{A}_2, A_2$ be disjoint liftings of $\tilde{A}, A$ with $\tilde{A}_1 \overset{sv}{\approx}_\varepsilon A_1$ and $\tilde{A}_2 \overset{sv}{\approx}_\varepsilon A_2$. By disjoint, we mean that no entry in some row and some column is non-zero in both A_1 and A_2 (and the same goes for $\tilde{A}_1$ and $\tilde{A}_2$).

Then, $\tilde{A}_1 + \tilde{A}_2 \overset{sv}{\approx}_\varepsilon A_1 + A_2$ because

$$\begin{aligned} |x^\top ((\tilde{A}_1 + \tilde{A}_2) - (A_1 + A_2))y| &\leq |x^\top (\tilde{A}_1 - A_1)y| + |x^\top (\tilde{A}_2 - A_2)y| \\ &\leq \frac{\varepsilon}{4} (x^\top (D_{\text{out},1} - A_1 D_{\text{in},1}^+ A_1^\top) x + y^\top (D_{\text{in},1} - A_1^\top D_{\text{out},1}^+ A_1) y) + \\ &\quad \frac{\varepsilon}{4} (x^\top (D_{\text{out},2} - A_2 D_{\text{in},2}^+ A_2^\top) x + y^\top (D_{\text{in},2} - A_2^\top D_{\text{out},2}^+ A_2) y). \end{aligned}$$

The final expression can be simplified by using the disjointness of the liftings (and thus the degree matrices).

Taking the tensor product of any $M \in \{0, 1\}^{N \times N}$ with $\tilde{A}, A$ is simply summing some number of disjoint liftings of $\tilde{A}, A$. So, $M \otimes \tilde{A} \overset{sv}{\approx}_\varepsilon M \otimes A$. $\square$

The random walk matrix of an expander graph H approximate the random walk matrix of the complete graph on $|V(H)|$ vertices, with an error ε that is linearly related to the expansion parameter $\lambda(H)$. This intuitively makes sense as expander graphs are meant to be highly “well-connected” and there is no more “well-connected” graph than the complete graph.

Formalizing this property relies on a weaker form of matrix approximation than SV approximation known as unit-circle approximation.

Definition 6.9. Let W and $\tilde{W}$ be doubly-stochastic matrices in $\mathbb{R}^{n \times n}$. Then, $\tilde{W}$ is a unit-circle ε -approximation of W if for all vectors $x, y \in \mathbb{R}^n$, we have

$$|x^\top (\tilde{W} - W)y| \leq \frac{\varepsilon}{2} (||x||^2 + ||y||^2 - |x^\top J_N x + y^\top J_N y|)$$

Unit-circle approximations were introduced in [AKMPSV20]. This work helped to understand how fixed-length walks on graph products like the derandomized square product behave. With this as a goal in mind, they proved the following characterization of expander graphs.

Lemma 6.10 ([AKMPSV20]). *Let H be a connected (N, D) -regular graph. Then, H is an (N, D, λ) -expander graph if and only if $H \overset{\circ}{\approx}_\lambda J_N$.*

Assuming the above, we obtain the following lemma.

Lemma 6.11 ([APPSV23]). *Let H be a connected (N, D) -regular graph. Then, H is an (N, D, λ) -expander graph if and only if $H \overset{svn}{\approx}_{2\lambda} J_N$.*

Proof. Recall that J_N is the matrix with $1/N$ in every entry (i.e., J_N is the random walk matrix of the complete graph on N vertices with a self-loop on every vertex).

By definition, $H \overset{svn}{\approx}_{2\lambda} J_N$ if and only if

$$\begin{aligned} |x^\top (H - J_N)y| &\leq \frac{\lambda}{2} (||x||^2 - ||J_N^\top x||^2 + ||y||^2 - ||J_N y||^2) \\ &= \frac{\lambda}{2} (||x||^2 - x^\top J_N J_N^\top x + ||y||^2 - y^\top J_N^\top J_N y) \\ &= \frac{\lambda}{2} (||x||^2 - x^\top J_N x + ||y||^2 - y^\top J_N y) \\ &= \frac{\lambda}{2} (||x||^2 + ||y||^2 - |x^\top J_N x + y^\top J_N y|) \end{aligned}$$

This means that $H \overset{svn}{\approx}_{2\lambda} J_n$ if and only if $H \overset{\circ}{\approx}_\lambda J_n$ in terms of the unit-circle approximation. However, $G \overset{\circ}{\approx}_\lambda J_n$ if and only if $\lambda(G) \leq \lambda$ by Lemma 6.10. $\square$

7 Fooling Permutation Branching Programs

To fool an arbitrary-width permutation branching program B with a single accept state, we should consider its graph representation, which we also denote by B (abusing notation). We will consider what happens when we apply the ultimate transformation $\mathcal{T}_n(B, H)$.

Notice that B (the graph for the branching program) is not entirely a regular graph itself, as is required of all the graphs in the paper thus far, due to the first layer V_0 having no incoming edges and the final layer V_n having no outgoing edges. We can resolve this by adding “wraparound” or two edges (labeled 0 and 1) from the i -th vertex in the final layer to the i -th vertex in the first layer (for each i).

Furthermore, in order to perform the first iteration $(B \otimes H)^8$ in the transformation (and consequently all subsequent iterations), we need the degree of B to match the number of vertices in H . We can duplicate each edge of B by a factor of $D^{16}/2$ to resolve this, where the edge labels are simply made to be strings of length $16 \log(D)$ with the first bit being the original label.

Definition 7.1. Let D be an implicit parameter, which is a power of 2. For a permutation branching program, let B denote the corresponding layered acyclic graph outlined in Definition 1.3.

Denote by G the graph B with the appropriate “wraparound” from the final to first layer and the duplications of each edge by a factor of $D^{16}/2$.

Now, we will show that the final graph product $\mathcal{T}_n(G, H)$ that would be used in Reingold’s algorithm for G is an approximation of a much simpler tensor product $G^n \otimes J$. Notice that the power of G in this tensor product is n , which directly represent the n steps on B or G (from the start vertex to the final layer) that the output of the zig-zag PRG ultimately aims to emulate.

The following lemma is the key step in such an approximation and may prove to be useful for other contexts in which the zig-zag product appear.

Lemma 7.2. Let G be a (N, K) -regular graph and let H be a (K, D, λ) -expander graph. Then,

$$G \otimes H \stackrel{\text{svn}}{\approx}_{O(\lambda)} G \otimes J_K$$

Proof. By Lemma 5.3, we can write

$$G \otimes H = (I \otimes H)P(I \otimes H)$$

where P is the permutation matrix that corresponds to the map Rot_G .

Similarly, we can write

$$G \otimes J_K = (I_N \otimes J_K)P(I_N \otimes J_K).$$

To see why this is true, we argue by basis vectors again (as we did in Lemma 5.3). Since $J_K v = u_K$ for any $v \in \mathbb{R}^K$ (i.e., the complete graph mixes immediately), we have

$$\begin{aligned} (G \otimes J_K)e_{(v,k)} &= (G \otimes J_K)(e_v \otimes e_k) = (Ge_v) \otimes (J_K e_k) \\ &= \left(\frac{1}{K} \sum_{\ell=1}^K e_{w_\ell} \right) \otimes u_K = \frac{1}{K} \sum_{\ell=1}^K e_{w_\ell} \otimes u_K \end{aligned}$$

for all $v \in [N]$ and all $k \in [K]$, where $w_1, \dots, w_K$ are the neighbors of v in G .

On the other hand, if we let $(w_\ell, \ell) = \text{Rot}_G(v, \ell)$ for each $\ell \in [K]$, we have

$$\begin{aligned} (I_N \otimes J_K)P(I_N \otimes J_K)e_{(v,k)} &= (I_N \otimes J_K)P(I_N \otimes J_K)(e_v \otimes e_k) = (I_N \otimes J_K)P(e_v \otimes u_K) \\ &= (I_N \otimes J_K)P\left(\frac{1}{K} \sum_{\ell=1}^K e_v \otimes e_\ell\right) = \frac{1}{K} \sum_{\ell=1}^K (I_N \otimes J_K)P(e_v \otimes e_\ell) \\ &= \frac{1}{K} \sum_{\ell=1}^K (I_N \otimes J_K)Pe_{(v,\ell)} = \frac{1}{K} \sum_{\ell=1}^K (I_N \otimes J_K)e_{w_\ell, \ell} \\ &= \frac{1}{K} \sum_{\ell=1}^K (I_N \otimes J_K)(e_{w_\ell} \otimes e_\ell) = \frac{1}{K} \sum_{\ell=1}^K e_{w_\ell} \otimes u_K. \end{aligned}$$

Two NK -by- NK matrices are equivalent as operators if and only if they act the same on the complete set of basis vectors $\{e_{(v,k)} : v \in [N], k \in [K]\}$. Thus, $G \otimes J_K = (I_N \otimes J_K)P(I_N \otimes J_K)$.

Now, by Lemmas 6.11 and 6.8, we get

$$I_N \otimes H \overset{svn}{\approx}_{2\lambda} I_N \otimes J_K.$$

Then, by Fact 6.7, we get

$$P(I_N \otimes H) \overset{svn}{\approx}_{2\lambda} P(I_N \otimes J_K).$$

Finally, by Fact 6.6, we get

$$(I_N \otimes H)P(I_N \otimes H) \overset{svn}{\approx}_{2\lambda} (I_N \otimes J_K)P(I_N \otimes J_K).$$

So, $G \otimes H \overset{svn}{\approx}_{O(\lambda)} G \otimes J_K$, as desired. $\square$

Theorem 7.3. *Let G be a (N, D^{16}) -regular graph and H be a (D^{16}, D, λ) -expander graph. Then,*

$$\mathcal{T}_n(G, H) \overset{svn}{\approx}_{\delta} G^n \otimes J$$

where $\delta = O(\lambda \log n)$. Recall that J is the random walk matrix of an appropriately-sized complete graph with a self-loop at every vertex.

Proof. Let $K = D^{16}$, the number of vertices in H and the degree of G . Throughout this proof, let $G_k = \mathcal{T}^k(G, H)$, for simplicity.

By Lemma 7.2, for any $k \in \mathbb{N}$, we can write

$$G_{k-1} \otimes H \overset{svn}{\approx}_{O(\lambda)} G_{k-1} \otimes J_K.$$

However, we need to take the eighth power of these graphs now. We know $(G_{k-1} \otimes J)^8 = G_{k-1}^8 \otimes J$ because we have

$$(G_{k-1}^t \otimes J_K)(G_{k-1} \otimes J_K) = (G_{k-1}^t G) \otimes (J_K J_K) = G_{k-1}^{t+1} \otimes J_K$$

for all $t \in \mathbb{N}$.

Moreover, Fact 6.7 implies that SV approximation is closed under powering graphs a constant number of times (up to a constant factor). Since $G_k = (G_{k-1} \otimes H)^8$ by definition, we get

$$G_k \overset{svn}{\approx}_{O(\lambda)} G_{k-1}^8 \otimes J_K.$$

If we repeatedly do this, we find

$$\begin{aligned} \mathcal{T}_n(G, H) &= G_{\log_8(n)} \overset{svn}{\approx}_{O(\lambda)} G_{\log_8(n)-1}^8 \otimes J_M \overset{svn}{\approx}_{O(\lambda)} G_{\log_8(n)-2}^{64} \otimes J_M^{\otimes 2} \\ &\overset{svn}{\approx}_{O(\lambda)} \dots \overset{svn}{\approx}_{O(\lambda)} G_0^n \otimes J^{\otimes \log_8(n)} = G_0^n \otimes J. \end{aligned}$$

Here, $J_K^{\otimes i}$ represents J_K tensored with itself i times. It is equal to an appropriately-sized matrix J itself.

This approximation is exactly what we wanted as we can then apply the quasi-triangle inequality from Fact 6.5. $\square$

Before we proceed to the final analysis of our zig-zag PRG, we need a good expander graph H to couple with G . The Margulis-Gabber-Galil expanders [GG81] are a class of strongly explicit expanders that will prove useful for this specific task.

Theorem 7.4. *There exists some positive real number $\lambda_* < 1$ and a strongly explicit family $\mathcal{H}$ of $(n^2, 8, \lambda_*)$ -expander graphs with consistent one-way labelings.*

The graph is constructed with the abelian group $\mathbb{Z}_n \times \mathbb{Z}_n$ as the vertex set, where each vertex (a, b) is connected to the vertices

$$(a+1, b), (a-1, b), (a, b+1), (a, b-1), (a, a+b), (a, a-b), (a+b, b), (a-b, b)$$

Such a graph can be naturally equipped with a consistent one-way labeling because each vertex's outgoing edges are based on invertible operations. Recall that powering a graph with a consistent one-way labeling yield another graph with a consistent one-way labeling.

Powering these graphs will improve the spectral expansion parameter properties of these graphs (since $\lambda_* < 1$) at the sacrifice of greater degree. Concretely, this is captured with the following.

Corollary 7.5. *For all constants $\lambda \in [0, 1)$, there exists some $D = \text{poly}(1/\lambda)$ such that there is an explicitly computable (D^{16}, D, λ) -expander graph H with a consistent one-way labeling.*

Proof. Let $D = 2^{3t}$, where t is an integer parameter based on n and ε to be chosen later. Since D^{16} is a perfect square, we can take an $(D^{16}, 8, \lambda_*)$ -expander graph H_0 from the explicit family guaranteed by Theorem 7.4. Note that $\lambda_* < 1$ is a constant.

Take the t -th power of the graph H_0 , defining it as $H = H_0^t$. Then, H is a $(D^{16}, 2^{3t}, \lambda_*^t)$ -expander graph. If we pick t to be the unique integer such that

$$\frac{\log(\lambda)}{\log(\lambda_*)} \leq t \leq \frac{\log(\lambda)}{\log(\lambda_*)} + 1,$$

then $\lambda_*^t \leq \lambda$. Since $D = 2^{3t}$, H is a (D^{16}, D, λ) -expander graph.

Finally, D is not too large as required. Since λ_* is some constant smaller than 1, we have

$$D = 2^{3t} \leq \lambda^{3/\log(\lambda_*)} \cdot 2^3 \leq O\left(\frac{1}{\lambda}\right)^{3/\log(1/\lambda_*)} \leq \text{poly}\left(\frac{1}{\lambda}\right). \quad \square$$

The main theorem now follows from what we have discussed so far by applying Theorem 7.3 to G and this expander graph H .

Proof of Theorem 1.10. By Corollary 7.5, we are guaranteed a (D^{16}, D, λ) -expander graph H , where $\lambda = O(\varepsilon/\log n)$ and $D = \text{poly}((\log n)/\varepsilon)$, which is computable in $\text{polylog}(D)$ time and $O(\log(D))$ space.

Let G be the graph representation of an arbitrary width- w length- n permutation branching program B with a single accept state, with the proper “wraparound” and edge duplication as in Definition 7.1. Let s denote the vertex which is the start state of the program, and let t denote the vertex which the accept state of the program.

By Theorem 7.3, assuming the appropriate implicit constant was chosen in our choice of λ , we have

$$\mathcal{T}_n(G, H) \stackrel{svn}{\approx}_{2\varepsilon} G^n \otimes J.$$

If V is the vertex set of G , then let $V \times A$ denote the vertex set of $\mathcal{T}_n(G, H)$ (and thus of $G^n \otimes J$). Let u be the vector in $\mathbb{R}^{V \times A}$ that is uniform on $\{s\} \times A$, let v be the vector in $\mathbb{R}^{V \times A}$ that is the indicator of $\{t\} \times A$. Define $x = |A|^{1/2} \cdot u$ and $y = |A|^{-1/2} \cdot v$ to be the normalizations of these vectors in $\mathbb{R}^{V \times A}$ with the standard Euclidean norm.

Now, we compare the quantities $u^\top (\mathcal{T}_n(G, H))v$ and $u^\top (G^n \otimes J)v$.

- $u^\top (\mathcal{T}_n(G, H))v$ is the probability that a random step starting uniformly at random in $\{s\} \times A$ on $\mathcal{T}_n(G, H)$ ends in $\{t\} \times A$. So, it is also the probability that the permutation branching program B accepts a *pseudorandom* string arising from a uniform random step on $\mathcal{T}_n(G, H)$.
- $u^\top (G^n \otimes J)v$ is the probability that a random step starting uniformly at random in $\{s\} \times A$ on $G^n \otimes J$ ends in $\{t\} \times A$. So, it is also the probability that the permutation branching program B accepts a *uniform random* string arising from a uniform random step on $G^n \otimes J$.

So, we want to bound $u^\top (\mathcal{T}_n(G, H) - G^n \otimes J)v$ above by ε . Since x and y are normalized vectors, we have

$$\begin{aligned} |u^\top (\mathcal{T}_n(G, H) - G^n \otimes J)v| &= |x^\top (\mathcal{T}_n(G, H) - G^n \otimes J)y| \\ &\leq \frac{2\varepsilon}{4} (\|x\|^2 - \|(G^n \otimes J)^\top x\|^2 + \|y\|^2 - \|(G^n \otimes J)y\|^2) \\ &\leq \frac{\varepsilon}{2} (1 - 0 + 1 - 0) = \varepsilon. \end{aligned}$$

Finally, by Fact 4.5, the seed length of GEN_n is $O(\log n \log D)$ or $O(\log n \cdot (\log \log n + \log(1/\varepsilon)))$. $\square$

8 Comparison to the INW Generator

Recall that the original proof of Theorem 1.10 involved the INW generator [HPV21].

Definition 8.1. Let $\mathcal{H}$ be a family of graphs such that, for each $k \in \mathbb{N}$, there is some $H_k \in \mathcal{H}$ which is a $(2D^k, D)$ -regular graph with a one-way labeling. Define $\text{INW}_k : \{0, 1\} \times [D]^k \rightarrow \{0, 1\}^{2^k}$ inductively in the following way.

- For $k = 0$, $\text{INW}_0(a) = a$
- For $k \in \mathbb{N}_0$, $\text{INW}_{k+1}(v, e) = (\text{INW}_k(v), \text{INW}_k(H_{k+1}[v, e]))$.

Then, the INW generator is $\text{INW}_{O(\log n)}$ for some appropriate implicit constant.

A good way to analyze the INW generator is to make use of another graph product called the derandomized square product [RV05].

Definition 8.2. Let G be a (N, K) -regular graph with a consistent one-way labeling, and let H be a (K, D) -regular graph with a one-way labeling. Then, the derandomized square product $G \mathbin{\text{\textcircled{S}}} H$ is defined to be the (N, KD) -regular graph with

$$(G \mathbin{\text{\textcircled{S}}} H)[v, (i, j)] = G[G[v, i], H[i, j]]$$

Lemma 8.3. Let G be a (N, D) -regular graph with a consistent one-way labeling, and let H be a (K, D) -regular graph with a one-way labeling. Then, $G \mathbin{\text{\textcircled{S}}} H$ has a consistent one-way labeling.

Proof. By definition, we have $(G \mathbin{\text{\textcircled{S}}} H)[v, (i, j)] = G[G[v, i], H[i, j]]$. To prove that $G \mathbin{\text{\textcircled{S}}} H$ has a consistent one-way labeling, fix (i, j) and suppose that $(G \mathbin{\text{\textcircled{S}}} H)[u, (i, j)] = (G \mathbin{\text{\textcircled{S}}} H)[v, (i, j)]$.

It suffices to show that $u = v$. Indeed, we have

$$G[G[u, i], H[i, j]] = G[G[v, i], H[i, j]]$$

As G has a consistent one-way labeling, we get $G[u, i] = G[v, i]$. Again, using the fact that G has a consistent one-way labeling, this implies that $u = v$, as desired. $\square$

We can formulate a statement similar to Theorem 5.7 for the INW generator in the context of a graph product called the derandomized square product.

Definition 8.4. Let G be an (N, D^{16}) -regular graph with a consistent one-way labeling and let $\mathcal{H}$ be a family of graphs such that, for each $k \in \mathbb{N}$, there is some $H_k \in \mathcal{H}$ which is a $(2D^k, D)$ -regular graph with a one-way labeling. We define a sequence $\{\mathcal{U}^k(G, H) : k \in \mathbb{N}_0\}$ inductively in the following way:

- $\mathcal{U}^0(G, H) = G$
- $\mathcal{U}^k(G, H) = \mathcal{U}^{k-1}(G, H) \mathbin{\text{\textcircled{S}}} H_k$ for each $k \in \mathbb{N}$

Theorem 8.5. Let G and H be graphs as in Definition 8.4. Then, for all $k \in \mathbb{N}_0$, we have

$$\mathcal{U}^k(G, H)[v, e] = G^{2^k}[v, \text{INW}_k(e)]$$

On the surface, the INW generator and the zig-zag generator are different. For instance, the INW generator uses a whole sequence of expander graphs whereas the zig-zag generator uses just one expander.

Additionally, most of the seed for the zig-zag generator comes from sampling a uniform random auxiliary vertex on the final graph product $\mathcal{T}_n(G, H)$ while the seed for the INW generator is entirely composed of a random step on the final graph product $\mathcal{U}^{O(\log n)}(G, H)$. The vertex set grows with repeated applications of the zig-zag product while the degree stays constant: the opposite is true of the derandomized square product.

However, the question of whether it is possible that the two generators could be unified in some way is also worth considering. The relationship between the generators may lie in an already-known relationship between the derandomized square and zig-zag products, also due to Rozenman and Vadhan [RV05].

Definition 8.6. Let G be a (NK, D) -regular graph on vertex set. The projection graph $P(G)$ is a (N, KD) -regular graph, where

$$P(G)[v, (a, i)] = G[(v, a), i]_1$$

where the right-hand side refers to the first component of the a -th neighbor of (v, x) in G . Note that labeling above is not necessarily consistent.

Proposition 8.7. *Let G be a (N, K) -regular graph, and let H be a (K, D) -regular graph. Then, the projection graph $P((G \otimes H)^2)$ is equal to the graph obtained from $(G \otimes H)^2$ by duplicating each edge K^2 times.*

Given this, it could be the case that the INW generator is a special case or projection of the zig-zag generator. Further research is required.

References

- [AKMPSV20] AmirMahdi Ahmadinejad, Jonathan Kelner, Jack Murtagh, John Peebles, Aaron Sidford, and Salil Vadhan. “High-precision Estimation of Random Walks in Small Space”. In: *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*. 2020, pp. 1295–1306. DOI: [10.1109/FOCS46700.2020.00123](https://doi.org/10.1109/FOCS46700.2020.00123).
- [APPSV23] AmirMahdi Ahmadinejad, John Peebles, Edward Pyne, Aaron Sidford, and Salil Vadhan. “Singular Value Approximation and Sparsifying Random Walks on Directed Graphs”. In: *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*. 2023, pp. 846–854. DOI: [10.1109/FOCS57990.2023.00054](https://doi.org/10.1109/FOCS57990.2023.00054).
- [Alo21] Noga Alon. “Explicit Expanders of Every Degree and Size”. In: *Combinatorica* 41.4 (Aug. 2021), pp. 447–463. ISSN: 1439-6912. DOI: [10.1007/s00493-020-4429-x](https://doi.org/10.1007/s00493-020-4429-x).
- [AS00] Noga Alon and Benny Sudakov. “Bipartite Subgraphs and the Smallest Eigenvalue”. In: *Combinatorics, Probability and Computing* 9.1 (2000), pp. 1–12. DOI: [10.1017/S0963548399004071](https://doi.org/10.1017/S0963548399004071).
- [BRRY14] Mark Braverman, Anup Rao, Ran Raz, and Amir Yehudayoff. “Pseudorandom Generators for Regular Branching Programs”. In: *SIAM Journal on Computing* 43.3 (2014), pp. 973–986. DOI: [10.1137/120875673](https://doi.org/10.1137/120875673). URL: <https://doi.org/10.1137/120875673>.
- [CKPPRSV17] Michael B. Cohen, Jonathan Kelner, John Peebles, Richard Peng, Anup B. Rao, Aaron Sidford, and Adrian Vladu. “Almost-linear-time algorithms for Markov chains and new spectral primitives for directed graphs”. In: *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*. STOC 2017. Montreal, Canada: Association for Computing Machinery, 2017, pp. 410–419. ISBN: 9781450345286. DOI: [10.1145/3055399.3055463](https://doi.org/10.1145/3055399.3055463). URL: <https://doi.org/10.1145/3055399.3055463>.
- [GG81] Ofer Gabber and Zvi Galil. “Explicit constructions of linear-sized superconcentrators”. In: *Journal of Computer and System Sciences* 22.3 (1981), pp. 407–420. ISSN: 0022-0000. DOI: [https://doi.org/10.1016/0022-0000\(81\)90040-4](https://doi.org/10.1016/0022-0000(81)90040-4). URL: <https://www.sciencedirect.com/science/article/pii/0022000081900404>.
- [HPV21] William M. Hoza, Edward Pyne, and Salil Vadhan. “Pseudorandom Generators for Unbounded-Width Permutation Branching Programs”. In: *12th Innovations in Theoretical Computer Science Conference (ITCS 2021)*. Ed. by James R. Lee. Vol. 185. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 7:1–7:20. ISBN: 978-3-95977-177-1. DOI: [10.4230/LIPIcs.ITCS.2021.7](https://doi.org/10.4230/LIPIcs.ITCS.2021.7). URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITCS.2021.7>.
- [INW94] Russell Impagliazzo, Noam Nisan, and Avi Wigderson. “Pseudorandomness for network algorithms”. In: *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*. STOC ’94. Montreal, Quebec, Canada: Association for Computing Machinery, 1994, pp. 356–364. ISBN: 0897916638. DOI: [10.1145/195058.195190](https://doi.org/10.1145/195058.195190). URL: <https://doi.org/10.1145/195058.195190>.
- [LPV23] Chin Ho Lee, Edward Pyne, and Salil Vadhan. “On the Power of Regular and Permutation Branching Programs”. In: *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2023)*. Ed. by Nicole Megow and Adam Smith. Vol. 275. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 44:1–44:22. ISBN: 978-3-95977-296-9. DOI: [10.4230/LIPIcs.APPROX/RANDOM.2023.44](https://doi.org/10.4230/LIPIcs.APPROX/RANDOM.2023.44). URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2023.44>.
- [Nis92] Noam Nisan. “Pseudorandom generators for space-bounded computation”. In: *Combinatorica* 12.4 (Dec. 1992), pp. 449–461. ISSN: 1439-6912. DOI: [10.1007/BF01305237](https://doi.org/10.1007/BF01305237). URL: <https://doi.org/10.1007/BF01305237>.

- [NZ96] Noam Nisan and David Zuckerman. “Randomness is Linear in Space”. In: *Journal of Computer and System Sciences* 52.1 (1996), pp. 43–52. ISSN: 0022-0000. DOI: <https://doi.org/10.1006/jcss.1996.0004>. URL: <https://www.sciencedirect.com/science/article/pii/S0022000096900045>.
- [Rei08] Omer Reingold. “Undirected connectivity in log-space”. In: *J. ACM* 55.4 (2008), 17:1–17:24. ISSN: 0004-5411. DOI: [10.1145/1391289.1391291](https://doi.org/10.1145/1391289.1391291).
- [RTV06] Omer Reingold, Luca Trevisan, and Salil Vadhan. “Pseudorandom walks on regular digraphs and the RL vs. L problem”. In: *Proceedings of the Thirty-Eighth Annual ACM Symposium on Theory of Computing*. STOC ’06. Seattle, WA, USA: Association for Computing Machinery, 2006, pp. 457–466. ISBN: 1595931341. DOI: [10.1145/1132516.1132583](https://doi.org/10.1145/1132516.1132583). URL: <https://doi.org/10.1145/1132516.1132583>.
- [RVW00] Omer Reingold, Salil Vadhan, and Avi Wigderson. “Entropy waves, the zig-zag graph product, and new constant-degree expanders and extractors”. In: *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*. FOCS ’00. USA: IEEE Computer Society, 2000, p. 3. ISBN: 0769508502.
- [RV05] Eyal Rozenman and Salil Vadhan. “Derandomized Squaring of Graphs”. In: *Approximation, Randomization and Combinatorial Optimization. Algorithms and Techniques*. Ed. by Chandra Chekuri, Klaus Jansen, José D. P. Rolim, and Luca Trevisan. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 436–447. ISBN: 978-3-540-31874-3.